INF 4040a Technologies for Web-based Information Systems

Project B - Support Ticketing System Sprint 2

Team 1

Viktor Logodazhki, ID# 200190541

Nikolay Pugyov, ID# 200190116

Department of Computer Science, AUBG

Blagoevgrad, 2026

1. Introduction

This document describes the work completed during Sprint 2 of the Support Ticketing System (STS) project for the course.

The objective of Sprint 2 was to extend the initial web application from Sprint 1 with responsive user interface improvements and the first ticket-management functionality. During this sprint, we focused on making the pages usable on both desktop and mobile devices, allowing authenticated users to create support tickets, and showing logged-in users the most recent tickets for their team on the home page.

The implemented functionality includes:

- Integrating Bootstrap into the application and updating the pages to behave responsively on desktop and mobile devices
- Displaying a shared top navigation menu on all pages, with links shown according to the current user's authentication status
- Allowing logged-in users to add new support tickets with subject, description, team, and status
- Showing logged-in users the 10 most recently added tickets for their team on the home page

These features build on the authentication and navigation foundation completed in Sprint 1 and introduce the core ticket workflow of the system.

2. System Architecture and Technologies

In Sprint 2, we continued developing STS as a web-based client-server application. Users access the system through a web browser, which renders the interface, sends HTTP requests to the server, and displays the returned views. The server side is implemented using ASP.NET Core MVC and is responsible for routing requests, validating input, executing the application logic, and communicating with the database.

The ticket-related data is stored in a MySQL database. Entity Framework Core is used to map the database tables to C# models and to perform the required queries and updates. During Sprint 2, the database schema was extended to support the ticket fields required for the sprint, including subject, team, status, and insertion timestamp.

The home page logic was also extended so that, when a user is logged in, the server determines the user's team and loads the 10 most recent tickets that belong to that team. These results are then passed to the Razor View and rendered as a list on the home screen.

On the client side, Bootstrap was integrated to support responsive page layouts, consistent spacing, form styling, and navigation behavior across desktop and mobile devices. A small JavaScript file was used to manage the mobile navigation toggle behavior.

The main technologies used in this sprint include:

- ASP.NET Core MVC - for building the web application and handling requests
- C# - for implementing the application logic
- MySQL - for storing ticket and user data
- Entity Framework Core - for managing database communication
- ASP.NET Identity - for maintaining authenticated user sessions
- Razor Views, HTML, CSS, JavaScript, Bootstrap - for building the user interface and responsive layout.

This architecture allowed us to implement the required functionality for Sprint 2 while preserving the structure established in Sprint 1.

3. Implemented User Stories

During Sprint 2, we implemented the three user stories defined in the project specification. These stories focused on responsive user interface behavior, ticket creation, and ticket visibility for authenticated users.

3.1 Story 1 - Responsive Pages and Shared Navigation

Implemented by: **Viktor Logodazhki**

User Story: As a user, I want to see all pages of the web app being responsive and visualize well on PC and mobile devices.

Implementation

This story required the application to use Bootstrap for rendering all pages, provide a top navigation menu on every page, and ensure that the layout adapts correctly to smaller screens such as phones and tablets. The links in the navigation also had to change depending on whether the visitor was anonymous or logged in.

To implement this functionality, Viktor updated the shared layout of the application so that all pages use a common Bootstrap-based header and navigation bar. Bootstrap is loaded from a CDN, and the menu contents are generated conditionally based on the visitor's authentication status.

Here is a shorter snippet from the shared layout showing the navigation toggle and the conditional menu links:

```
<button class="navbar-toggler sts-site-nav__toggle" type="button"
data-sts-nav-toggle
        aria-controls="mainNav" aria-expanded="false"
aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
</button>

<div class="navbar-collapse sts-site-nav__collapse" id="mainNav"
aria-hidden="true">
    <div class="sts-site-nav__panel">
        <ul class="navbar-nav ms-auto align-items-lg-center gap-lg-2
sts-site-nav__list">
            <li class="nav-item sts-site-nav__item">
                <a asp-controller="Home" asp-action="Index"
class="nav-link sts-site-nav__link">Home</a>
```

```
            </li>
            @if (User.Identity?.IsAuthenticated ?? false)
            {
                <li class="nav-item sts-site-nav__item">
                    <a asp-controller="Ticket" asp-action="Add"
class="nav-link sts-site-nav__link">Add
                        New</a>
                </li>
                <li class="nav-item sts-site-nav__item">
                    <form asp-controller="Account"
asp-action="Logout" method="post"
                          class="m-0 d-grid sts-site-nav__form">
                        <button type="submit" class="btn
btn-outline-secondary btn-sm sts-site-nav__action">
                            Logout
                        </button>
                    </form>
                </li>
            }
            else
            {
                <li class="nav-item sts-site-nav__item">
                    <a asp-controller="Account" asp-action="Register"
class="nav-link sts-site-nav__link">Register</a>
                </li>
                <li class="nav-item sts-site-nav__item">
                    <a asp-controller="Account" asp-action="Login"
                       class="btn btn-primary btn-sm
sts-site-nav__action">Login</a>
                </li>
            }
        </ul>
    </div>
</div>
```

In order to make the navigation work cleanly on smaller screens, a custom JavaScript menu toggle was added. The following excerpt shows the menu state handling:

```
const desktopMedia = window.matchMedia("(min-width: 992px)");
const openClass = "sts-site-nav--menu-open";
const closeTargets = ".sts-site-nav__link, .sts-site-nav__action";
```

```
const render = () => {
  const isOpen = !desktopMedia.matches &&
nav.classList.contains(openClass);
  nav.classList.toggle(openClass, isOpen);
  toggle.setAttribute("aria-expanded", String(isOpen));

  if (desktopMedia.matches) {
    menu.removeAttribute("aria-hidden");
    return;
  }

  menu.setAttribute("aria-hidden", String(!isOpen));
};

const closeMenu = () => {
  nav.classList.remove(openClass);
  render();
};

toggle.addEventListener("click", () => {
  if (desktopMedia.matches) {
    return;
  }

  nav.classList.toggle(openClass);
  render();
});
```

As a result, all pages now share one top navigation structure, the links are shown according to the current authentication state, and the layout remains usable across both desktop and mobile screen sizes without requiring horizontal scrolling.

3.2 Story 2 - Add Support Tickets

Implemented by: **Viktor Logodazhki and Nikolay Pugyov**

User Story: As a logged-in user, I want to be able to add support tickets to the system.

Implementation

This story required authenticated users to be able to create a support ticket by entering a subject, description, team, and status. The system also had to validate the entered data, reject invalid values, and redirect the user to the home page after a successful submission. The implementation of this story was shared between both team members.

Viktor focused on the user-facing part of the feature. He implemented the ticket entry page, the Bootstrap-based form layout, the select elements for team and status, and the display of validation messages when invalid input is submitted.

The ticket entry form is defined in the Razor View as follows:

```
<form asp-action="Add" method="post" novalidate>
    <div asp-validation-summary="ModelOnly" class="alert alert-danger
validation-summary-valid" role="alert"></div>

    <div class="mb-3">
        <label asp-for="Subject" class="form-label"></label>
        <input asp-for="Subject" class="form-control" />
        <span asp-validation-for="Subject" class="invalid-feedback
d-block"></span>
    </div>

    <div class="row g-3">
        <div class="col-md-6">
            <label asp-for="Team" class="form-label"></label>
            <select asp-for="Team" class="form-select">
                <option value="">-- Select Team --</option>
                <option value="Development">Development</option>
                <option value="Support">Support</option>
                <option value="Sales">Sales</option>
            </select>
            <span asp-validation-for="Team" class="invalid-feedback
d-block"></span>
        </div>
        <div class="col-md-6">
            <label asp-for="Status" class="form-label"></label>
            <select asp-for="Status" class="form-select">
                <option value="">-- Select Status --</option>
```

```
                <option value="New">New</option>
                <option value="Open">Open</option>
                <option value="Closed">Closed</option>
            </select>
            <span asp-validation-for="Status" class="invalid-feedback
d-block"></span>
        </div>
    </div>
</form>
```

Viktor also contributed to the validation rules by using a dedicated view model together with custom validation attributes. The CreateTicketViewModel ensures that the required fields are present, while the AllowedTeam and AllowedTicketStatus attributes guarantee that only valid team and status values are accepted.

```
[Required(ErrorMessage = "Subject is required.")]
[StringLength(200, ErrorMessage = "Subject must be at most 200
characters long.")]
public string Subject { get; set; } = string.Empty;

[AllowedTeam]
public string Team { get; set; } = string.Empty;

[AllowedTicketStatus]
public string Status { get; set; } = string.Empty;
```

Nikolay focused on the server-side workflow of the feature. He implemented the authenticated controller actions, ensured that the current user is identified before a ticket is saved, added the service logic for creating tickets, and handled the redirect back to the home page after a successful submission.

Here is a shorter excerpt from the controller action that processes the ticket entry:

```
ModelState.Clear();
if (!TryValidateModel(model))
{
    return View(model);
}
```

```
var user = await _userManager.GetUserAsync(User);
if (user is null)
{
    return Challenge();
}

var result = await _ticketService.CreateAsync(new CreateTicketRequest
{
    Subject = model.Subject,
    Description = model.Description,
    Team = team,
    Status = status,
    CreatedByUserId = user.Id
});

if (!result.Succeeded)
{
    foreach (var (key, errors) in result.Errors)
    {
        foreach (var error in errors)
        {
            ModelState.AddModelError(key, error);
        }
    }

    return View(model);
}

return RedirectToAction("Index", "Home");
```

The persistence logic is implemented in the ticket service. It creates a new Ticket entity, sets the insertion timestamp, and saves the record through Entity Framework Core.

```
var ticket = new Ticket
{
    Subject = request.Subject,
    Description = string.IsNullOrWhiteSpace(request.Description) ?
null : request.Description,
    Team = request.Team,
    Status = request.Status,
    CreatedByUserId = request.CreatedByUserId,
```

```
    CreatedAtUtc = DateTime.UtcNow
};

_dbContext.Tickets.Add(ticket);

try
{
    await _dbContext.SaveChangesAsync();
    return TicketCreationResult.Success();
}
catch (DbUpdateException)
{
    return TicketCreationResult.Failed((string.Empty, "The ticket
could not be saved. Please try again."));
}
```

During Sprint 2, the ticket schema was also updated through a migration so that the database contains the fields required for this functionality, including the Subject and Team columns.

Overall, this story introduced the first complete ticket-entry workflow in the application, from the UI form to validation, persistence, and redirect behavior.

3.3 Story 3 - Display Recent Tickets for the User's Team

Implemented by: **Nikolay Pugyov**

User Story: As a logged-in user, I want to see the 10 most-recently added tickets for my team.

Implementation

This story required the home screen for authenticated users to be updated so that it displays the 10 most recently inserted tickets belonging to the current user's team. The list had to show the ticket subject and team, and the entries had to be ordered from newest to oldest.

To implement this functionality, Nikolay extended the HomeController so that, when the current user is authenticated, the application loads the user's team and requests the 10 newest tickets for that team from the ticket service.

```
if (!model.IsAuthenticated)
{
    return View(model);
}

var user = await _userManager.GetUserAsync(User);
if (user is null)
{
    model.IsAuthenticated = false;
    return View(model);
}

model.CurrentTeam = user.Team;
model.RecentTickets = await
_ticketService.GetRecentTicketsForTeamAsync(user.Team, 10);
return View(model);
```

The actual filtering, ordering, and limiting logic is implemented in the ticket service. The query returns only tickets that belong to the current team, orders them by insertion time in descending order, and limits the result to the newest 10 entries.

```
public async Task<IReadOnlyList<RecentTicketListItem>>
GetRecentTicketsForTeamAsync(Team team, int maxCount)
{
    return await _dbContext.Tickets
        .AsNoTracking()
        .Where(ticket => ticket.Team == team)
        .OrderByDescending(ticket => ticket.CreatedAtUtc)
        .Take(maxCount)
        .Select(ticket => new RecentTicketListItem
        {
            Subject = ticket.Subject,
            Team = ticket.Team,
            CreatedAtUtc = ticket.CreatedAtUtc
        })
        .ToListAsync();
```

```
}
```

The Razor View was then updated so that authenticated users see a section on the home page containing the latest tickets for their team. The following excerpt shows the heading and the rendered ticket items:

```
@if (Model.IsAuthenticated)
{
    <div
        class="d-flex flex-column flex-md-row justify-content-between
align-items-md-center gap-3 mb-3">
        <div>
            <h2 class="h4 mb-1 sts-home__tickets-heading">Latest
Tickets for @Model.CurrentTeam</h2>
            <p class="text-secondary mb-0
sts-home__tickets-copy">Showing the 10 most recently added
                tickets for your team.</p>
        </div>
    </div>

    <div class="list-group list-group-flush sts-home__ticket-list">
        @foreach (var ticket in Model.RecentTickets)
        {
            <div class="list-group-item px-0 py-3
sts-home__ticket-item">
                <div
                    class="d-flex flex-column flex-sm-row
justify-content-between align-items-sm-center gap-2">
                    <div>
                        <h3 class="h6 mb-1">@ticket.Subject</h3>
                        <span class="badge text-bg-light
border">@ticket.Team</span>
                    </div>
                    <small
                        class="text-secondary
sts-home__timestamp">@ticket.CreatedAtUtc.ToLocalTime().ToString("g")
</small>
                </div>
            </div>
        }
    </div>
```

```
}
```

This logic ensures that logged-in users immediately see the most relevant ticket information for their own team after entering the system.

4. Testing and Validation

Here, we present the acceptance tests performed for the user stories implemented in Sprint 2. Each test describes the input actions, the expected system behavior, and the observed result.

4.1 Test for Story 1 - Desktop Responsive Layout and Shared Navigation

Test Description

This test verifies that when the application is opened on a desktop-sized screen, the pages are rendered with the shared top navigation bar and the content is properly aligned.

Input

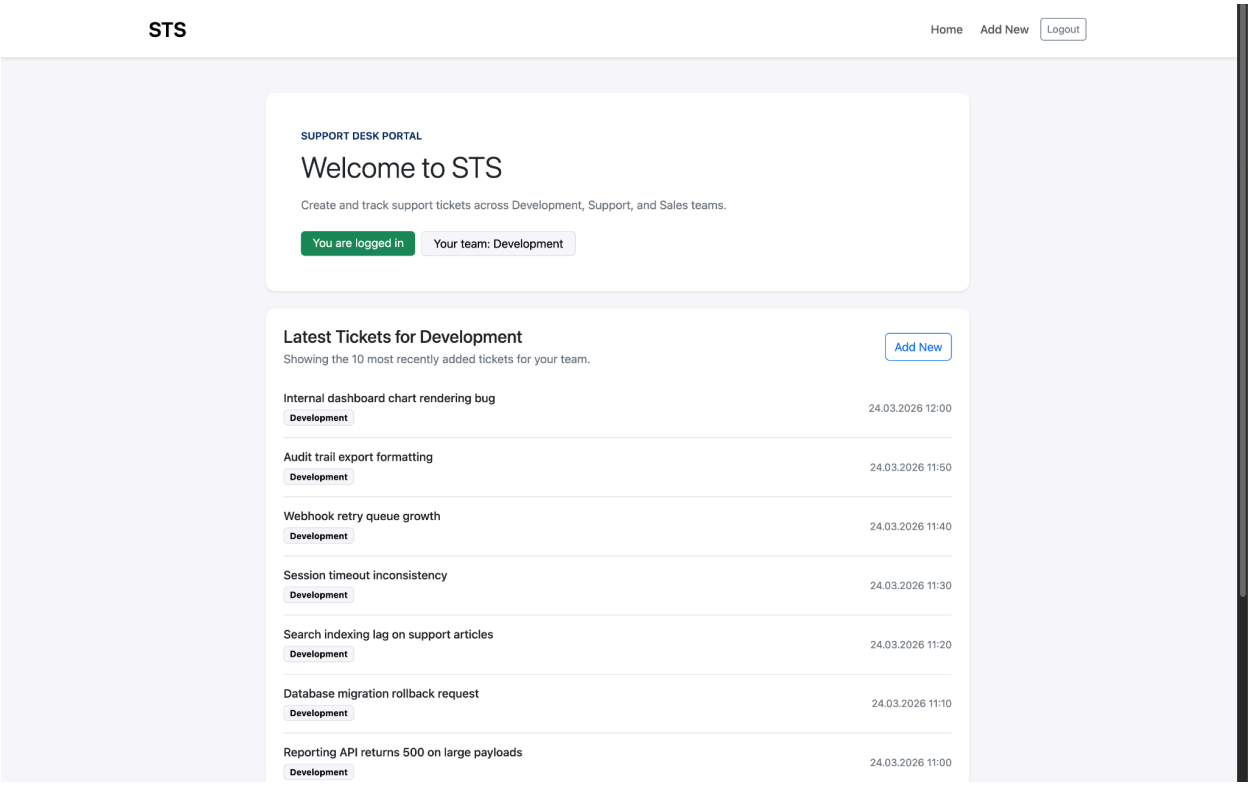
The user opens the application in a desktop web browser and navigates through the available pages.

Expected Output

The system should display a top navigation menu on all pages. Anonymous users should see the Register and Login links, while authenticated users should see the Home, Add New, and Logout actions. Page elements should be aligned correctly and should remain within the visible viewport.

Result

The desktop layout was rendered correctly. The top navigation menu was visible across pages, the links changed according to authentication status, and the content was aligned properly.



4.2 Test for Story 1 - Mobile Navigation and Small-Screen Responsiveness

Test Description

This test verifies that the application remains usable on a smaller screen and that the navigation menu behaves correctly on mobile devices.

Input

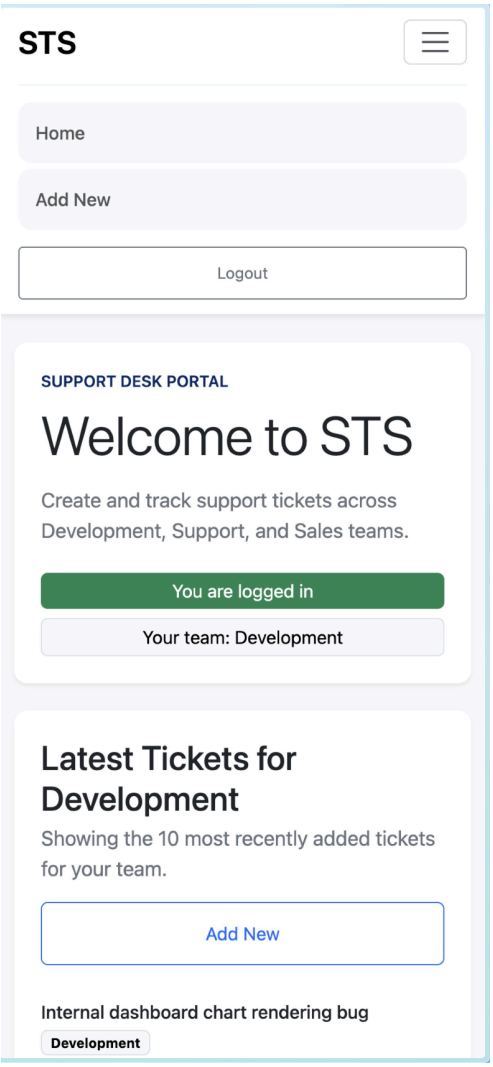
The user opens the application on a mobile-sized browser window and uses the navigation toggle.

Expected Output

The system should display a responsive layout without horizontal scrolling. The navigation should collapse into a mobile menu, and the available links should remain visible and usable according to the visitor's authentication state.

Result

The mobile layout was displayed correctly. The menu toggle opened and closed the navigation panel, the links remained accessible, and no horizontal scroll appeared.



4.3 Test for Story 2 - Successful Ticket Creation

Test Description

This test verifies that a logged-in user can successfully create a new support ticket.

Input

The user logs in and opens the Add New page. The following ticket data is entered:

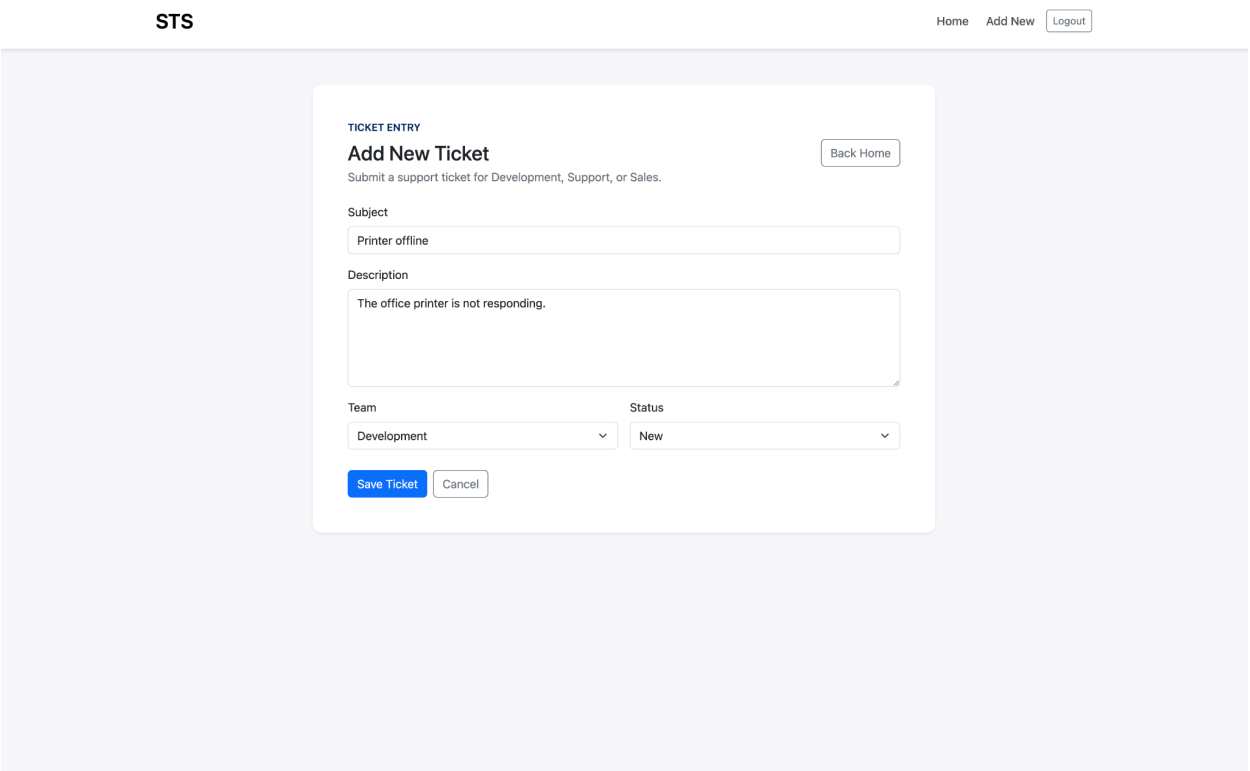
- Subject: Printer offline
- Description: The office printer is not responding.
- Team: Development
- Status: New

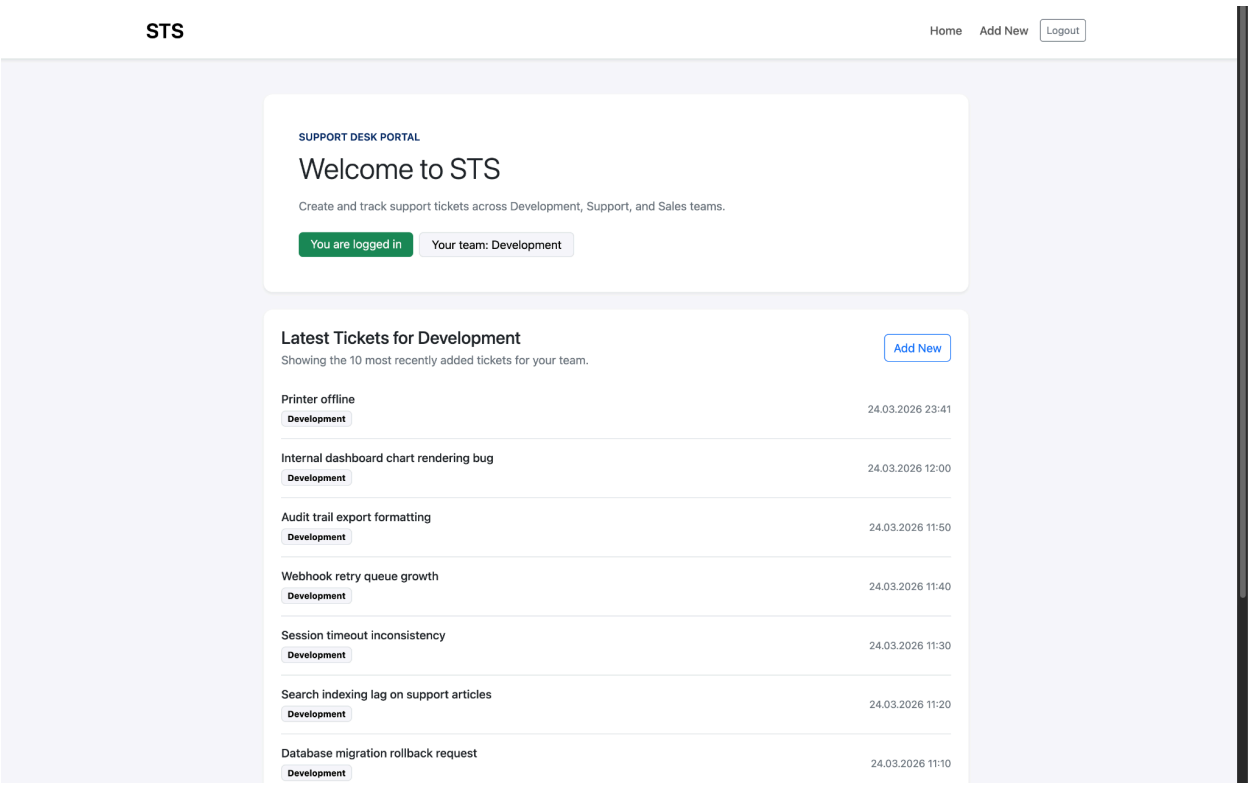
Expected Output

The system should validate the input, save the new ticket, and redirect the user to the home page.

Result

The ticket was successfully created and the user was redirected to the home page. The ticket then appeared in the recent ticket list for the user's team.





4.4 Test for Story 2 - Ticket Validation Errors

Test Description

This test verifies that invalid ticket input is rejected and appropriate validation messages are shown to the user.

Input

The logged-in user opens the Add New page and submits the form with invalid data, for example:

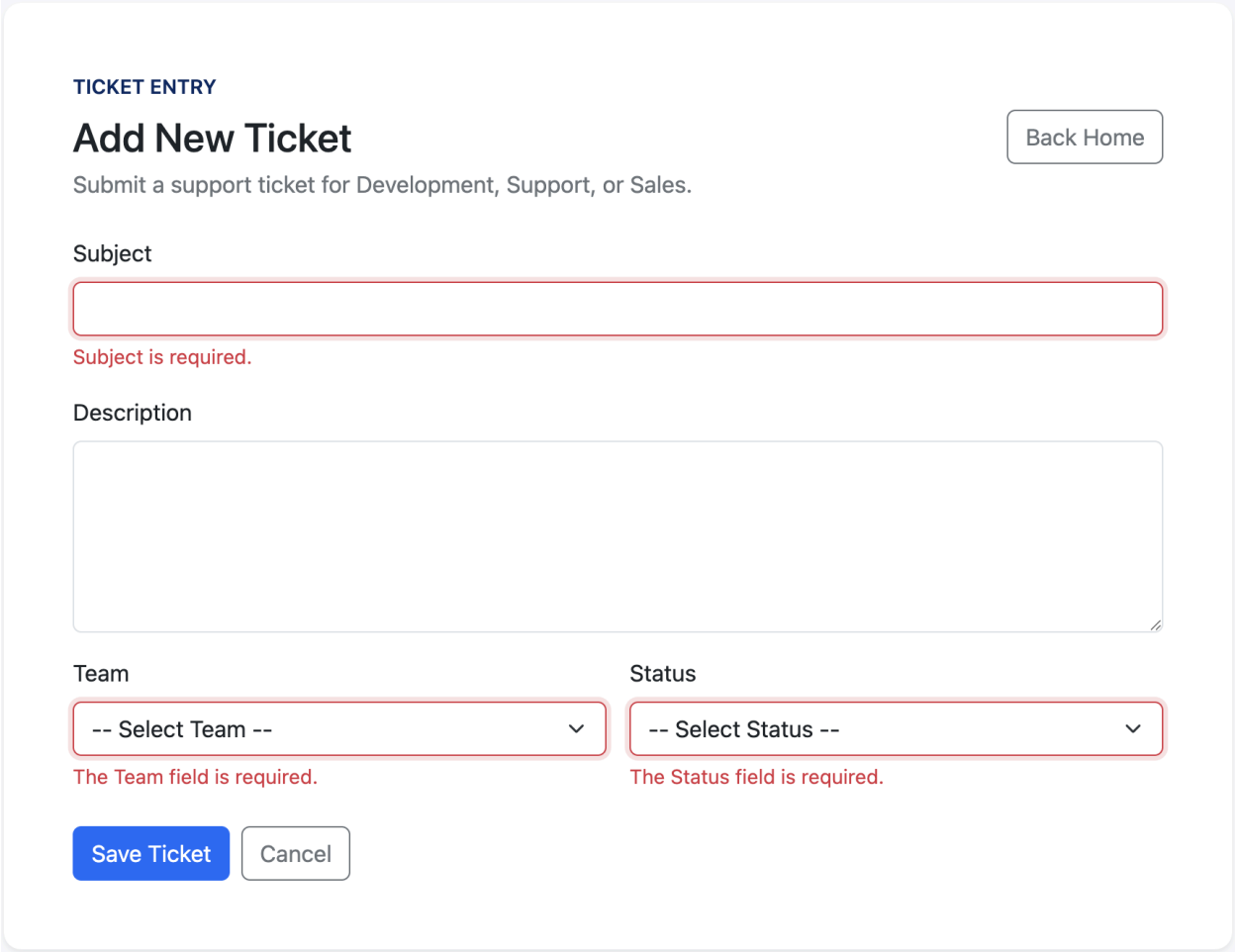
- Missing subject
- Missing team
- Invalid status value

Expected Output

The system should not save the ticket. The form should remain visible and the corresponding validation messages should be displayed to the user.

Result

The invalid ticket was rejected. The page remained open and the system displayed the validation errors correctly.



4.5 Test for Story 3 - Display Recent Tickets for the Current Team

Test Description

This test verifies that a logged-in user can see recently added tickets for their own team on the home page.

Input

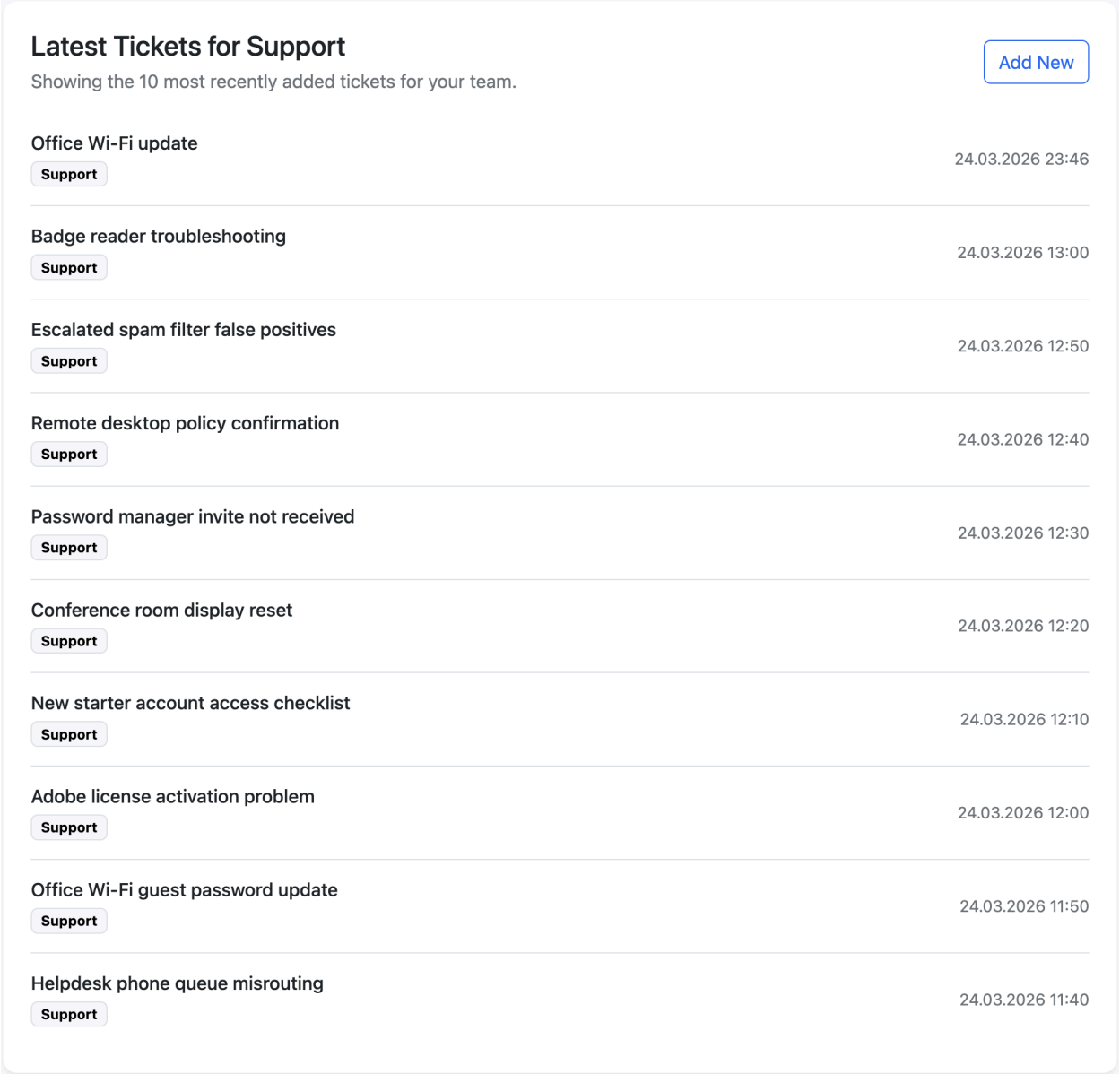
The user logs in after several tickets have already been created for the same team.

Expected Output

The system should display a recent-ticket section on the home page containing the subjects and team values of the newest tickets for that user's team.

Result

The home page displayed the recent-ticket section correctly and showed the newest tickets for the user's team.



4.6 Test for Story 3 - Filtering, Ordering, and 10-Item Limit

Test Description

This test verifies that the recent-ticket list contains only tickets for the current user's team, that the newest tickets are shown first, and that the list contains at most 10 entries.

Input

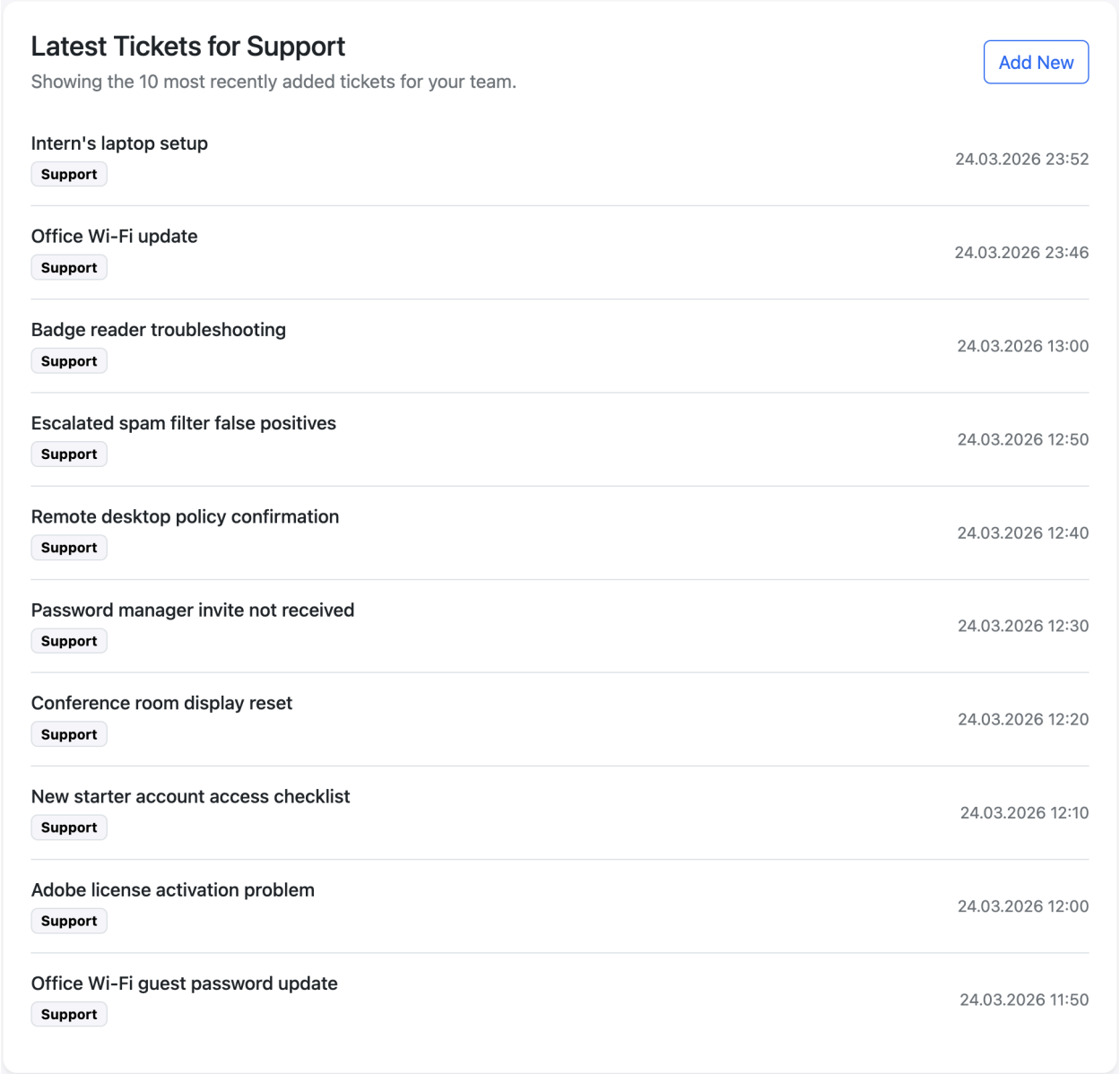
More than 10 tickets are created for one team, and at least one ticket is created for another team. The user then logs in as a member of the first team and opens the home page.

Expected Output

The system should display only the 10 most recent tickets for the user's team, ordered by insertion time in descending order. Tickets belonging to other teams should not appear.

Result

The home page displayed only the relevant team tickets, ordered from newest to oldest, and the list was limited to 10 entries.



Note: You can see the newest tickets are shown first, the last one dropped from the list

4.7 Automated Tests

In addition to the manual acceptance tests, we implemented automated unit and integration tests that cover functional Sprint 2 behavior. These tests verify ticket creation, recent-ticket retrieval, filtering by team, newest-first ordering, form validation, and authentication-related flow such as redirecting anonymous users away from the ticket entry page. Current automated test result: 52/52 tests passing.

5. Error Handling and Validation

In addition to the successful scenarios described in the acceptance tests, the system includes validation and error-handling logic to ensure that invalid input and invalid states are handled correctly. These checks prevent incorrect ticket data from being saved and provide feedback to the user.

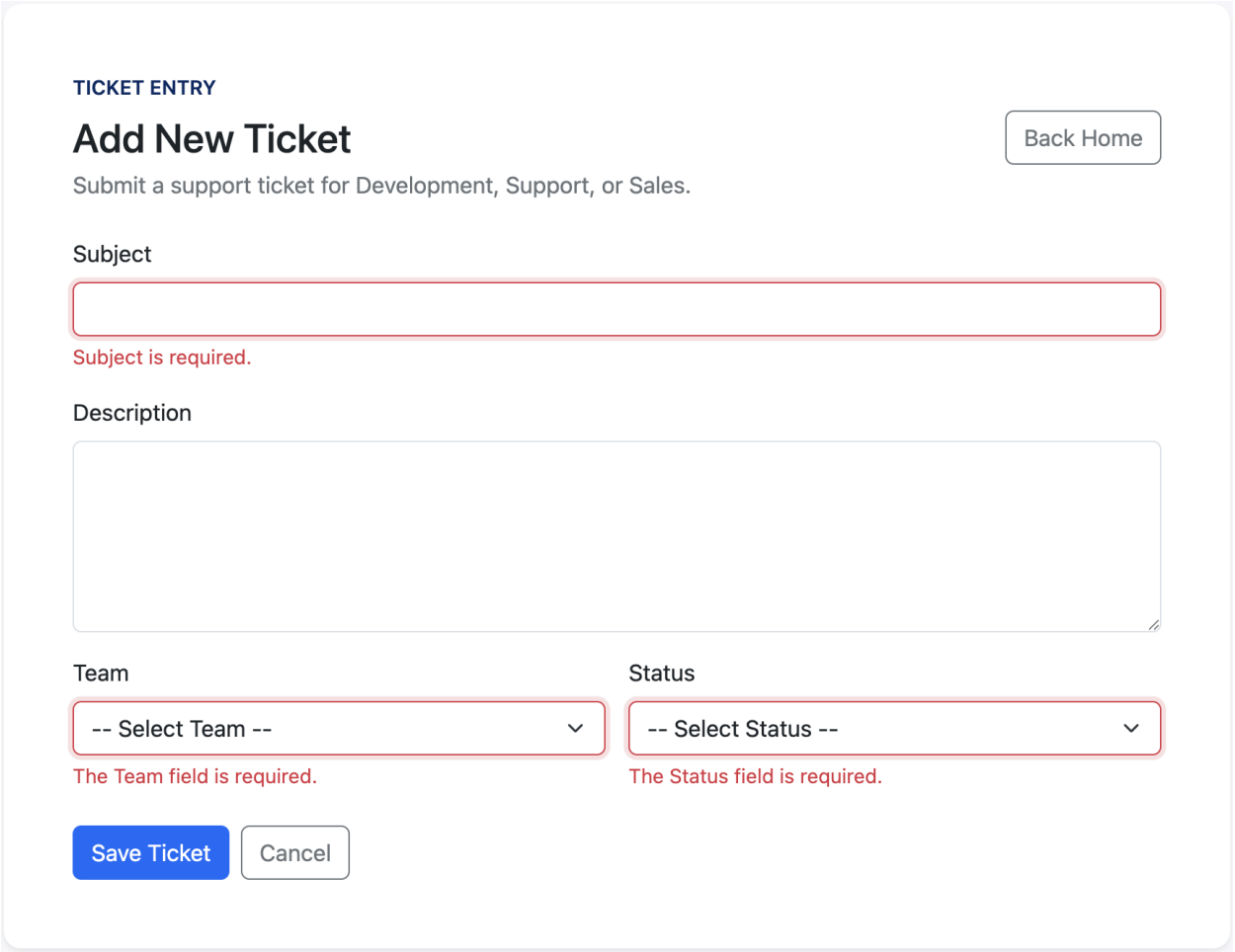
5.1 Ticket Entry Validation Errors

The Add New ticket form includes several validation checks before a ticket can be stored in the system.

The following validations were implemented and tested:

- missing subject
- missing team
- missing status
- invalid team value
- invalid status value
- description longer than the allowed maximum length

When invalid data is submitted, the ticket is not created and the form displays the corresponding validation messages, such as "Subject is required.", "Team is required.", and "Status must be one of: New, Open, Closed."



5.2 Authentication Protection for Ticket Entry

The ticket entry functionality is available only to authenticated users. The TicketController is protected with the [Authorize] attribute, which prevents anonymous visitors from accessing the page directly. If an anonymous user attempts to visit /Ticket/Add, the system redirects the user to the login page instead of showing the form.

https://localhost:7224/Ticket/Add

https://localhost:7224/Ticket/Add

https://localhost:7224/Ticket/Add

SUPPORT DESK PORTAL

Welcome to STS

Create and track support tickets across Development, Support, and Sales teams.

Create Account | Sign In

https://localhost:7224/Account/Login?ReturnUrl=%2FTicket%2FAdd

STS

Home | Register | Login

Login

Sign in with your STS email and password.

Email

Email is required.

Password

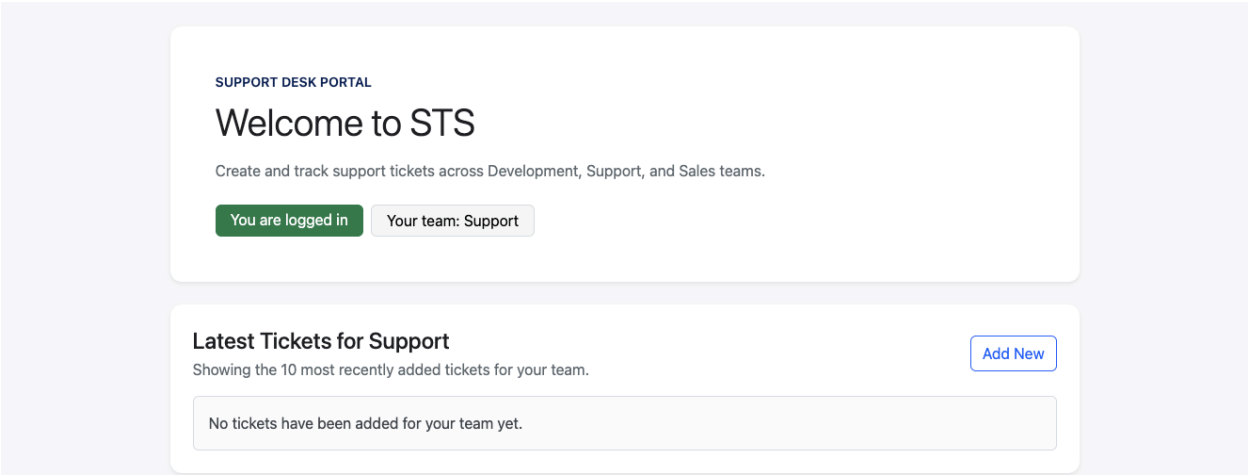
Login

5.3 Ticket Persistence Error Handling

The ticket service also includes application-level error handling around database persistence. If the system is unable to save a ticket successfully, the service returns a failure result and the controller adds the corresponding error messages to the model state so that they can be displayed to the user instead of failing silently.

5.4 Empty State for the Recent Ticket List

The recent-ticket section on the home page handles the case where no tickets have yet been created for the user's team. Instead of displaying an empty list, the system shows a message informing the user that no tickets have been added for the team yet. This improves the usability of the home page and ensures that the user still receives meaningful feedback in that case.



6. Team Collaboration and Conclusion

Although the Sprint 2 user stories were divided by primary responsibility, the development process remained collaborative and both team members contributed to the overall result.

Viktor was primarily responsible for Story 1, where he focused on the responsive Bootstrap-based interface, the shared navigation structure, and the improvements

needed for desktop and mobile usability. In Story 2, Viktor contributed the ticket-entry page, the form layout, and the user-facing validation and feedback behavior.

Nikolay was primarily responsible for Story 3, where he implemented the logic for loading and displaying the 10 most recent tickets for the current user's team. In Story 2, Nikolay contributed the controller and service workflow, ticket persistence, and the redirect behavior after successful ticket creation.

Despite this division of responsibilities, both of us collaborated on the design of the ticket workflow, the integration of the new homepage behavior, testing the completed stories, and preparing the project documentation. Sprint 2 significantly expanded the STS project from a basic authentication application into a responsive ticketing system with core ticket entry and team-based visibility features.